Movie Rec Hub: a Microservice Movie Recommendation Platform

PSIA Project Report - Group 34

Group members: number - Gonçalo Torcato, 120202 - Sara Nascimento, 120232 - Sofia Rodrigues

Video: link

1 Introduction

Movie Rec Hub is a web application where users browse movies, write reviews, organise movies into lists and receive personalised recommendations. The recommendations are produced by a large language model (LLM) that takes into account each user's lists, reviews and favourite genres.

The application is built as a set of Spring Boot microservices, each with its own PostgreSQL database. It runs locally with Docker Compose and in the Google Cloud on a Kubernetes cluster created with Terraform and configured with Ansible. The system also includes an LLM-based recommendation service and a monitoring stack with Prometheus and Grafana. As an advanced feature, the application keeps its state across a full shutdown of the infrastructure.

This report presents the design and deployment of the base system, including monitoring (Section 2), the persistent state feature (Section 3) and an evaluation of throughput and latency as the number of clients grows (Section 4). The video linked above shows the system running in the cloud.

2 System Design and Architecture

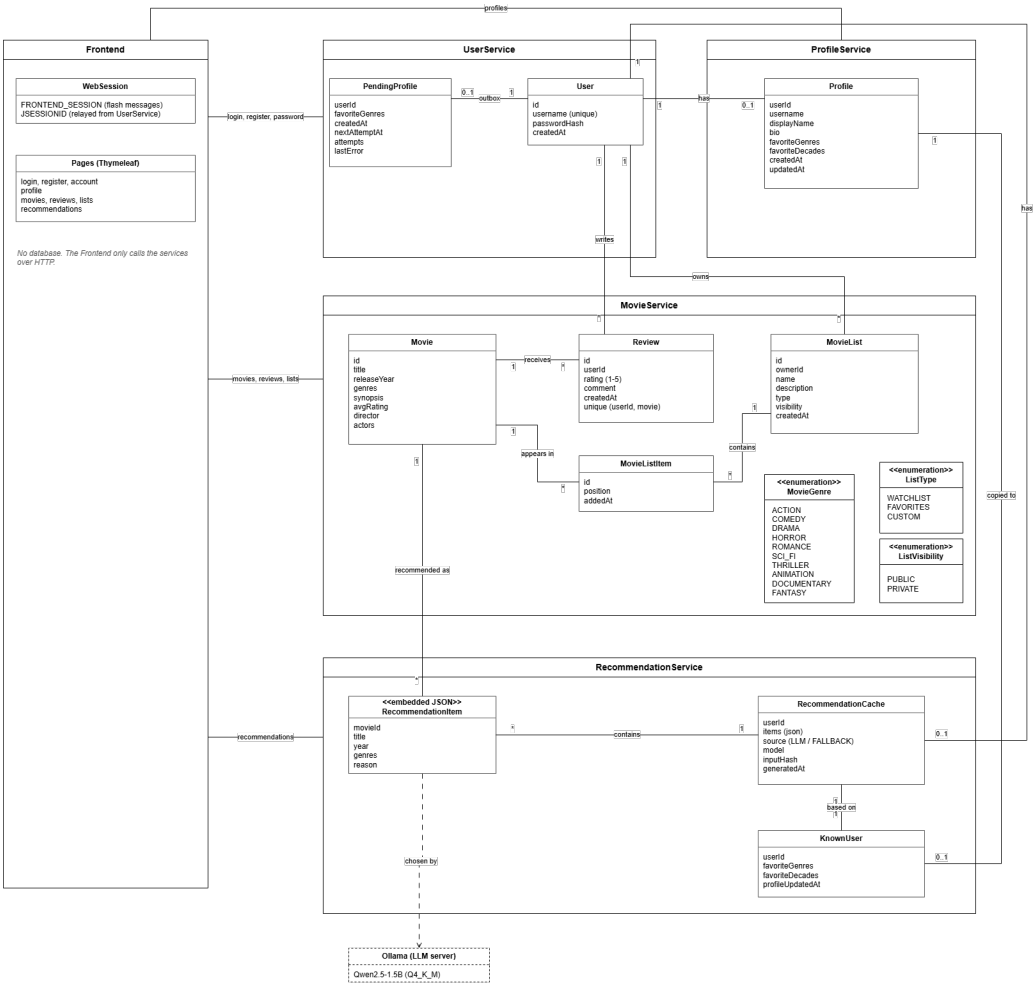


Fig. 1. Architecture of Movie Rec Hub.

The system is composed of five services that communicate through REST APIs:

- **Frontend:** the only entry point for users. It renders the web pages and calls the other services.
- **UserService:** accounts, login and sessions.
- **ProfileService:** user profiles, favourite genres and decades.
- **MovieService:** the movie catalogue, reviews and movie lists.
- **RecommendationService:** personalised recommendations using an LLM served by Ollama (Section 2.5).

Each service owns its database (users, profiles, movies and recommendations), so services never share tables and can be deployed and scaled independently.

2.1 Frontend

The Frontend is a Spring Boot application that renders pages on the server with Thymeleaf. A controller prepares the data in a Model and Thymeleaf fills an HTML template before it is sent to the browser, so the browser receives a complete page. Templates use `th:` attributes inside plain HTML: `th:if` shows a block only when a condition holds, `th:text` inserts values and `th:replace` includes reusable fragments, such as the sidebar, the alerts and the movie grid, defined once in a shared layout.

The Frontend talks to each backend service through a dedicated client class (for example, `MovieServiceClient`) with its own DTOs. The main pages are the movie catalogue with a genre filter, the movie details with reviews, the user's lists, login and registration, the profile pages and the "For you" recommendations page.

2.2 UserService and Testing

2.3 ProfileService and Testing

2.4 MovieService and Testing

The MovieService manages the movie catalogue, the reviews and the users' movie lists. It exposes a REST API under `/api/movies`, `/api/reviews` and `/api/lists` and stores its data in its own PostgreSQL database.

On first start, when the database is empty, the service fills the catalogue from The Movie Database (TMDB) API: it fetches the top-rated movies and stores, for each one, the title, release year, genres, synopsis, director and main actors.

The data model has four entities:

- **Movie:** title, release year, genres, synopsis, director, actors and the average rating.
- **Review:** a rating from 1 to 5 and a comment, written by a user about a movie. A user can review each movie only once (unique constraint on user and movie). Every time a review is created or deleted, the movie's average rating is recalculated.
- **MovieList:** a named list owned by a user, of type `WATCHLIST`, `FAVORITES` or `CUSTOM` and `PUBLIC` or `PRIVATE`. A user cannot have two lists with the same name.
- **MovieListItem:** a movie inside a list, with its position and the date it was added. The same movie cannot appear twice in one list.

The service enforces these rules itself. A private list is only visible to its owner and only the owner can change or delete a list. A global exception handler translates errors into clear HTTP codes. The API supports listing movies with a genre filter, searching by title, director or actor, filtering by year, creating and deleting reviews and creating, renaming, deleting and editing lists.

The MovieService follows a layered design: entities, repositories (Spring Data JPA), services with the business rules and REST controllers. It is covered by 156 automated tests in 10 test classes, each layer tested in isolation:

- **Controllers (53 tests):** `@WebMvcTest` with MockMvc, with the service layer replaced by mocks.
- **Services (53 tests):** JUnit 5 and Mockito, with the repositories mocked.
- **Repositories (39 tests):** `@DataJpaTest` on an in-memory H2 database, so no PostgreSQL instance is needed.
- **Entities (11 tests):** plain JUnit 5.

2.5 RecommendationService and Testing

The RecommendationService suggests movies in two ways: automatically, from what the user already likes and on demand, from a typed request such as "a drama movie". The choice is made by a small language model (Qwen2.5 1.5B) served by Ollama inside the same system, so no external AI service is used. It runs on CPU only, which limits its quality but keeps the deployment simple.

A background agent runs every 5 minutes and prepares 5 movies per user (the "For you" list), stored in the service's own PostgreSQL database; it skips users without signals and users whose taste has not changed (it compares a SHA-256 hash of the taste text). A typed request is sent to the model right away. Both are shown on the "For you" page of the Frontend.

The *taste* of a user is a short text built from the favourite genres and decades (ProfileService), the movie lists and the reviews (MovieService); movies already in lists or reviews are sent as "Already seen", so the model is asked not to repeat them. The prompt contains the whole catalogue (about 40 movies), the taste and the request and the model must answer in a fixed JSON format with movie ids. The service only checks the structure (ids exist, no duplicates, 1 to 5 movies).

The RecommendationService is covered by 66 automated tests in 10 test classes, each layer tested in isolation, without a database or Ollama:

- **Controller (8 tests):** `@WebMvcTest` with MockMvc, with the service layer replaced by mocks.
- **Services (33 tests):** JUnit 5 and Mockito, covering the recommendations, the background agent, the user's taste text, the known users and the catalogue refresh.
- **Engine (20 tests):** plain JUnit 5, covering the prompt builder, the validation of the model's answer and the taste hash.
- **Ollama client (5 tests):** Mockito, covering the request to the model and its errors.

Unit tests cannot judge the quality of a language model, so we also sent 22 typed requests to the real model (quantity, genre,

decade, combined, typos and prompt injection) before and after improving the prompt. Invalid answers dropped from 2 to 0 and requests with the right number of movies rose from 14 to 20. The model handles quantity, combined requests and typos well, but fails with decades (0 of 4) and some single-genre requests. We attribute this to the small quantized model running on CPU, since further prompt changes brought no gain.

2.6 Deployment

Each service is packaged as a Docker image, built in two stages: Maven compiles the application in a build image and only the resulting JAR is copied to a smaller Java runtime image. Locally, Docker Compose starts the whole system with one command: the five services, their four PostgreSQL databases, Ollama, Prometheus and Grafana.

COMPLETAR C A PARTE CLOUD

2.7 Monitoring with Prometheus and Grafana

All five services expose metrics at `/actuator/prometheus` using Spring Boot Actuator and Micrometer, which required only two dependencies and three configuration properties. One of them enables a histogram of HTTP request durations, so that Grafana can compute latency percentiles (p50 and p95) instead of only the average. The average hides slow requests: if 95 requests take 50 ms and 5 take 2 s, the average is about 150 ms, a value no user actually experienced. The p50 shows the typical user's latency and the p95 shows the delay felt by the slowest 5% of requests, which is where performance problems appear first.

Prometheus collects the metrics every 15 seconds and Grafana displays them. Both are configured from files in the repository (provisioning), so the same dashboards appear automatically locally and in the cluster. We built two dashboards:

- **Services:** it shows availability, requests per second, latency (p50, p95 and maximum), the slowest endpoints, the error rate (4xx and 5xx), JVM memory, CPU, threads and database connection pools.
- **Cluster:** adapted from a community dashboard (`https://github.com/dotdc/grafana-dashboards-kubernetes`), with a section we added for the application: pods not ready, restarts per pod and free space on the database disk. It also shows CPU and memory per node and per namespace.

PRINTS

Fig. 2. Services dashboard in Grafana.

In Kubernetes, the monitoring stack runs in its own namespace and is deployed with an Ansible playbook that follows the structure of the application's deployment playbook. Because pods change IP address whenever they are recreated, Prometheus discovers its targets through the Kubernetes API instead of using fixed addresses. It collects metrics from four sources (Fig. 3): the Spring services, node-exporter (one pod per VM, for CPU, memory and disks), cAdvisor (built into each node's kubelet, for each container) and kube-state-metrics (the state of Kubernetes objects, such as ready pods and restarts).

DIAGRAMA AQUI

Fig. 3. Monitoring architecture in the Kubernetes cluster.

Security was considered throughout. Prometheus and kube-state-metrics have read-only permissions and kube-state-metrics can only read pods, nodes and namespaces. Prometheus has no login, so it is reachable only inside the cluster; external access goes through Grafana, which requires a password and blocks sign-up and anonymous access. The Grafana password is kept out of Git and passed to Kubernetes as a Secret, without being written anywhere outside `deployment.yml`.

3 Advanced Components: Persistent State

4 Evaluation

4.1 Method

TO-DO - PRINTS E TAL

4.2 Results

Table 1. Throughput and latency for an increasing number of clients.

Clients	Throughput (req/s)	p50 latency (ms)	p95 latency (ms)	Errors (%)
1				
2				
4				
8				
16				
32				

5 Conclusion